A field that is a primary key must contain data values that are never repeated in the table.

The primary key can be a field that is already used, provided it is unique, for example the ISBN in the book table. The PATIENT table would need an extra field for each record as all of the existing fields could contain repeated data. To create a primary key, we could add a new field to each record, for example a unique number could be added to each patient's record. The extra field is:

- Primary key field called **`HospitalNumber`**

Activity 9.3

Using *Access* (or any other suitable database management software), add the **`HospitalNumber`** field to the single table database PATIENT.

- Choose 'text' as the data type for this field.
- Include validation checks to ensure that 8 characters must be entered starting with **`HN`** followed by **`6 digits`** for example **`HN123456`**.
- Choose suitable data to store in each primary key field for the 6 patients in the table.

Activity 9.4

1. Write down **four** database data types. Describe each data type and give an example of a field where it would be a suitable choice.
2. Choose suitable words/phrases from the following list to correctly complete the paragraph that follows:

Word list:	database	fields	records
	record	table	primary key
	data type	validation	text
	char	Boolean	field

You will need to use some words more than once.

A single .... contains one ..... .

Each ..... consists of many ..... . Every .....

has the same number of ..... . Every ..... is given a

..... . Examples of data types are ....., ...

and ..... . Some ..... will have ..... rules.

9.1.4 SQL

Structured Query Language (SQL) is the standard query language for writing scripts to obtain useful information from a database. We will be using SQL to obtain information from single-table databases. This will provide a basic understanding of how to obtain and display only the information required from a database. SQL is pronounced as es-queue-el.

For example, somebody needing to visit a patient would only require the ward number and the bed number of that patient in order to find where they are in the hospital. Whereas a consultant could need a list of the names of all the patients that they care for.

SQL scripts

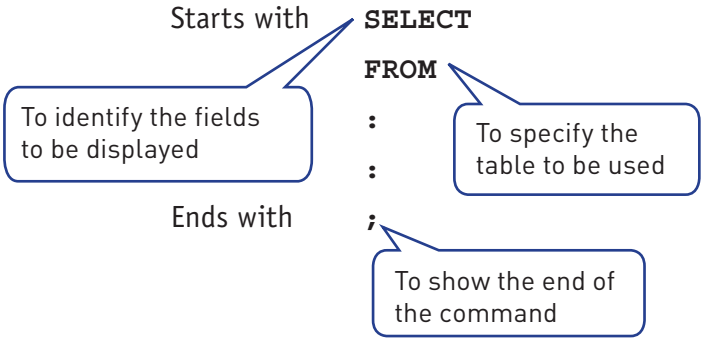
An **SQL script** is a list of SQL commands that perform a given task, often stored in a file so the script can be reused.

In order to be able to understand SQL and identify the output from an SQL script, you should have practical experience of writing SQL scripts. You can write scripts using SQL commands in *Access*. There are many other applications that also allow you to do this – *MySQL* and *SQLite* are freely available ones. When using any SQL application, it is important that you check the commands available to use as these may differ slightly from those listed in the syllabus and shown below.

You will need to be able to understand and identify the output from the following SQL statements.

SQL Query Statement	Description
`SELECT`	Fetches specified fields (columns) from a table; queries always begin with `SELECT`.
`FROM`	Identifies the table to use.
`WHERE`	Includes only records (rows) in a query that match a given condition.
`ORDER BY`	Sorts the results from a query by a given column either alphabetically or numerically.
`SUM`	Returns the sum of all the values in a field (column). Used with `SELECT`.
`COUNT`	Counts the number of records (rows) where the field (column) matches a specified condition. Used with `SELECT`.

An SQL command:



Only the `SELECT` and `FROM` commands are mandatory in an SQL script. All other commands are optional.

A **SELECT** statement takes the form:

`SELECT Field1, Field2, Field3`, etc. – this specifies the individual fields (columns) to be shown.

`SELECT *` – this specifies that **all** fields (columns) are to be shown.

A **FROM** statement takes the form:

FROM TableName – this specifies the table to use.

A **WHERE** statement takes the form:

WHERE Condition – this specifies the condition to apply.

Conditions often include values from fields, these values need to be stated in a form that matches the data type for the field.

Field type	Example value	General notes	Access notes
text	'Mr Smith'	Text field values should be in enclosed in single quotation marks.	Double quotation marks can also be used.
character	'M'	Character field values should be in enclosed in single quotation marks.	Double quotation marks can also be used.
Boolean	TRUE	Boolean can be **TRUE** or **FALSE**	Data type is Yes/No
integer	12	Integer field values should be whole numbers.	Allows integer or decimal values.
real	12.01	Real field values should be decimal numbers.	Allows integer or decimal values.
Date/time	'22/11/2022'	Date/time field values should be in enclosed in single quotation marks.	Date/time field values **must** be in enclosed in hashes (**#**).

Conditions also require operators to compare values from fields.

Operator	Description
=	equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than equal to
<>	not equal to
BETWEEN	between a range of two values
LIKE	search for a pattern
IN	specify multiple values
AND	specify multiple conditions that must all be true
OR	specify multiple conditions where one or more conditions must be true
NOT	specify a condition that must be false

An **ORDER BY** statement takes the form:

ORDER BY Field1, Field2, etc. – this specifies a sort in ascending or alphabetical order starting with the first field.

ORDER BY Field1, Field2 DESC – this specifies a sort in descending or reverse alphabetical order starting with the first field.

A **SUM** statement takes the form:

SELECT SUM (Field) – this specifies the field (column) for the calculation. The field should be integer or real.

A **COUNT** statement takes the form:

SELECT COUNT (Field) – this specifies the field (column) to count if the given criterium is met.

Example 1: Display consultant's patients

For example, the following SQL command for the PATIENT single-table database would provide a list of all Mr Smith's patients showing the hospital number, first name and family name for each of his patients.

```
SELECT HospitalNumber, FirstName, FamilyName
FROM PATIENT
WHERE Consultant = 'Mr Smith';
```

would display:

HospitalNumber	FirstName	FamilyName
HN123456	Winnie	Sing
HN123458	Min	Hoo

▲ **Figure 9.5** Output from the PATIENT table showing Mr Smith's patients

Example 2: Display consultant's patients in alphabetical order

This SQL command sorts the records in alphabetical order of family name:

```
SELECT HospitalNumber, FirstName, FamilyName
FROM PATIENT
WHERE Consultant = 'Mr Smith'
ORDER BY FamilyName;
```

would display:

HospitalNumber	FirstName	FamilyName
HN123458	Min	Hoo
HN123456	Winnie	Sing

▲ **Figure 9.6** Output from the PATIENT table showing Mr Smith's patients in alphabetical order of family name

Activity 9.5

1 Using the single table database PATIENT you have created.
   a Write an SQL query to list all Mr Jones' patients.
   b Write an SQL query to list all the patients not in ward 6.
   c Write an SQL query to list all the patients who arrived on 12/11/2022.
   d Write an SQL query to list all the patients who arrived between 12/10/2022 **AND** 30/10/2022.
2 Write down the output from this SQL query.

```
SELECT FirstName, FamilyName, BedNumber
FROM PATIENT
WHERE WardNumber = 7;
```

Activity 9.6

1 Using the single table database PATIENT you have created.
   a Write an SQL query to count the number of patients in ward 7.
   b Write an SQL query to count the number of patients not in ward 7.
2 Write down the output from this SQL query.

```
SELECT HospitalNumber, FirstName, FamilyName, Consultant
FROM PATIENT
ORDER BY Consultant, FamilyName;
```

Practical use of a database

As an IGCSE Computer Science student you need to be able to do the following:

- define a single-table database from given data storage requirements
- choose a suitable primary key for a database table
- read, complete and understand SQL scripts.

In order to do this, you will need to use a database management system. The following case study shows how to set up a database with *Microsoft Access 365* and complete the tasks described above.

Boys and girls between the ages of seven and eleven can join a cub scout group. (http://en.wikipedia.org/wiki/Cub_Scout). Each cub scout group needs to keep records about its members. Most groups will keep the following information about each cub in their group:

Activity 9.7

1 **a** Set up a cub scout database including appropriate validation checks for each field.

Enter data for at least 10 records.

b The cub scout leader wants to put each cub into a group called a 'six'; each 'six' can have up to six cubs in it and is given a name for example red, yellow, blue and green.

Add a new text field called Six, and assign each cub to either a red, blue, yellow or green six.

i Write an SQL query to pick out any cubs in the red six.

ii Write an SQL query to pick out any cubs in the red six or the blue six.

iii Write an SQL query to count the number of cubs in the red six.

c The cub scout leader wants to calculate the number of badges that all the cubs have been awarded. Add a new integer field called *Badges*, enter the number of badges awarded to each cub.

Write an SQL query to count the number of badges awarded to the whole cub group.

Activity 9.8

1 A database of students is to be set up with the following fields:

- Family name
- Other names
- Student ID
- Date of Birth
- Date of Entry to School
- Current Class
- Current school year/grade
- Email address.

a Select a data type for each field.

b Which fields should be validated, and which fields should be verified?

c Decide the validation rules for those fields which should be validated.

d Which field would you choose for the primary key?

e Choose a suitable format for the student ID.

f Build a database with at least 10 records; include all your validation checks. Ensure there are at least 3 different classes and 2 different years/grades.

g Set up and test SQL scripts to:

i Display Other names, Family and Email address in alphabetical order of family name.

ii Select all the students from each class in alphabetical order.

iii Select all the students for each year/grade and print Other names, Family name and Date of Birth, grouping the students by class.

Link

For more on verification, see Chapter 7.